

random_forest and boosting

```
#Libraries  
library(GGally)
```

Loading required package: ggplot2

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(randomForest)
```

randomForest 4.7-1.2

Type rfNews() to see new features/changes/bug fixes.

Attaching package: 'randomForest'

The following object is masked from 'package:dplyr':

combine

The following object is masked from 'package:ggplot2':

margin

```
library(gbm)
```

Loaded gbm 2.2.3

This version of gbm is no longer under development. Consider transitioning to gbm3, <https://github.com/josh1b/gbm3>

```
library(caret)
```

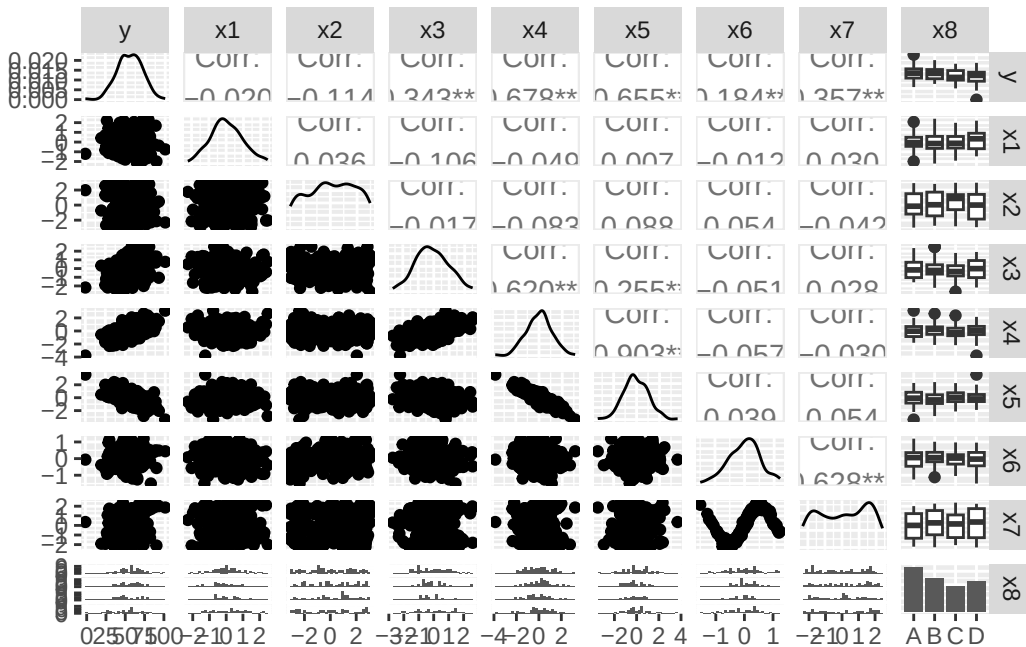
Loading required package: lattice

Data Setup

```
df <- read.csv("data/FP_Data.csv")
df <- df %>%
  mutate(across(where(is.character), as.factor))
train_df <- df[1:160,]
test_df <- df[-(1:160),]
```

```
df %>%
  ggpairs()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Random Forest Numeric

```
set.seed(28)
bag.fp <- randomForest(y ~ ., data = train_df, mtry = 8, importance = TRUE)
bag.fp
```

Call:

```
randomForest(formula = y ~ ., data = train_df, mtry = 8, importance = TRUE)
```

Type of random forest: regression

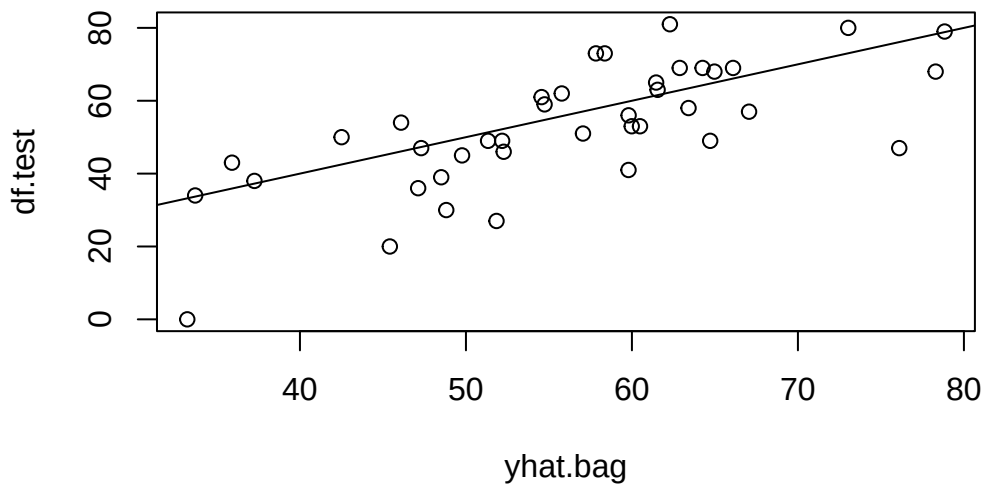
Number of trees: 500

No. of variables tried at each split: 8

Mean of squared residuals: 104.4786

% Var explained: 54.02

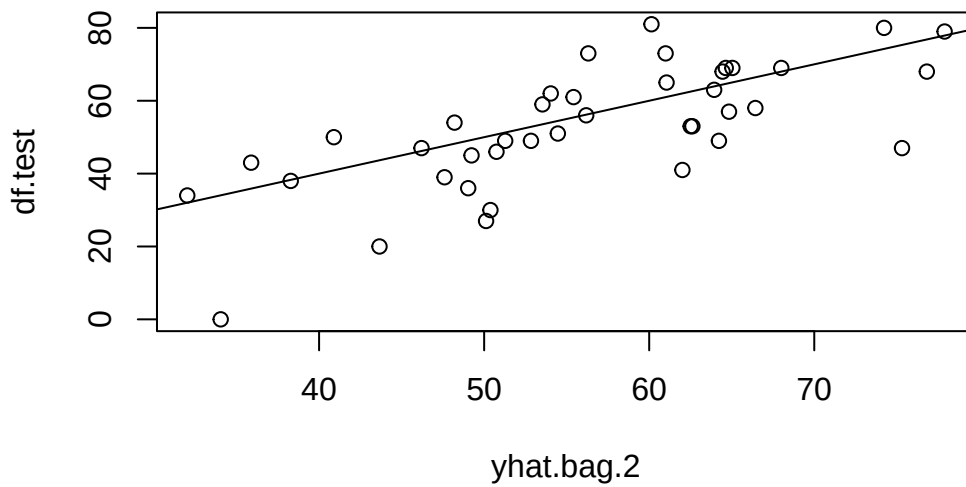
```
yhat.bag <- predict(bag.fp, newdata = test_df)
df.test <- test_df$y
plot(yhat.bag, df.test)
abline(0,1)
```



```
mean((yhat.bag - df.test)^2)
```

```
[1] 151.3738
```

```
bag.fp.2 <- randomForest(y ~ ., data = train_df, mtry = 8, ntree = 25)
yhat.bag.2 <- predict(bag.fp.2, newdata = test_df)
plot(yhat.bag.2, df.test)
abline(0,1)
```



```
mean((yhat.bag.2 - df.test)^2)
```

```
[1] 152.6574
```

```
rf.df <- randomForest(y ~ ., data = train_df, mtry = 6, importance = TRUE)
rf.df
```

Call:

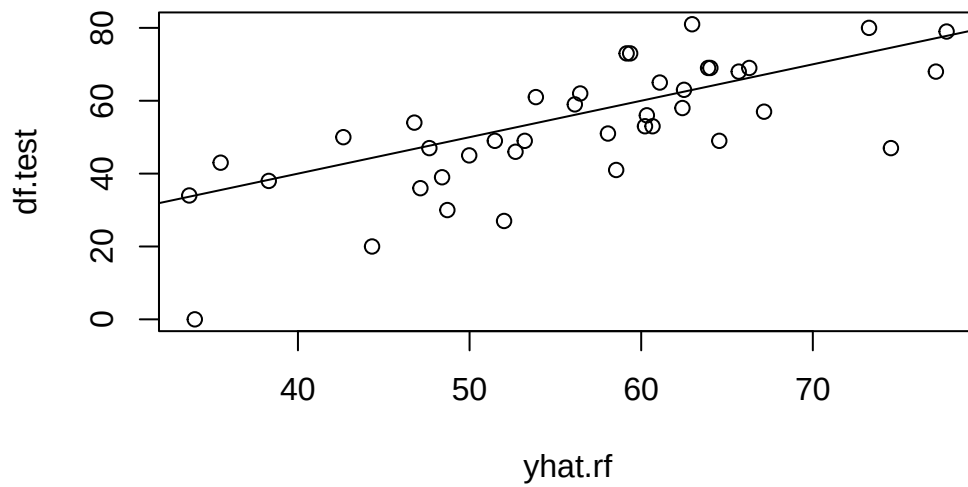
```
randomForest(formula = y ~ ., data = train_df, mtry = 6, importance = TRUE)
      Type of random forest: regression
      Number of trees: 500
```

No. of variables tried at each split: 6

Mean of squared residuals: 103.2981

% Var explained: 54.54

```
yhat.rf <- predict(rf.df, newdata = test_df)
plot(yhat.rf, df.test)
abline(0,1)
```



```
mean((yhat.rf - df.test)^2)
```

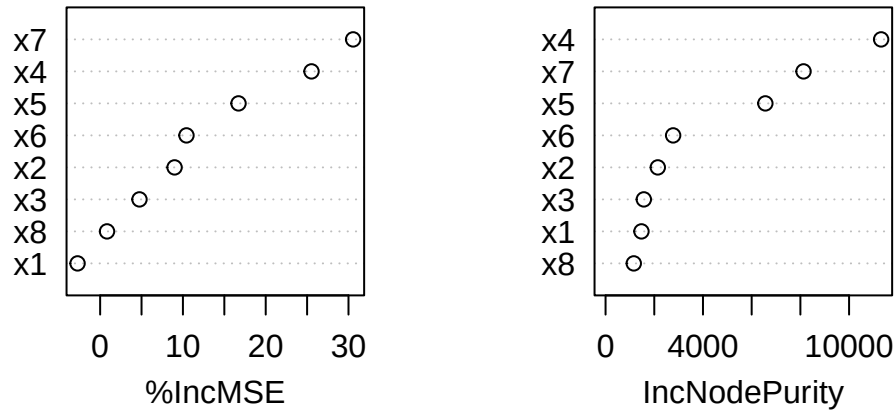
```
[1] 145.1044
```

```
importance(rf.df)
```

	%IncMSE	IncNodePurity
x1	-2.7271640	1473.561
x2	8.9891165	2144.729
x3	4.7501967	1570.585
x4	25.5300079	11315.831
x5	16.7139826	6561.290
x6	10.4277691	2774.224
x7	30.5584703	8129.273
x8	0.8297963	1158.755

```
varImpPlot(rf.df)
```

rf.df



Random Forest Categorical

```
df_cat <- df %>%  
  mutate(ybin = if_else(y >= 70, 1, 0))  
df_cat <- df_cat %>%  
  select(-y)  
df_cat_train <- df_cat[1:160,]  
df_cat_test <- df_cat[-(1:160),]  
bin.test <- df_cat_test$ybin  
rf.df.bin <- randomForest(ybin ~ ., data = df_cat_train, mtry = 6, importance = TRUE)
```

Warning in randomForest.default(m, y, ...): The response has five or fewer unique values. Are you sure you want to do regression?

rf.df.bin

Call:

```
randomForest(formula = ybin ~ ., data = df_cat_train, mtry = 6, importance = TRUE)  
Type of random forest: regression
```

Number of trees: 500
No. of variables tried at each split: 6

Mean of squared residuals: 0.1122321
% Var explained: 29.85

```
yhat.rf.bin <- predict(rf.df.bin, newdata = df_cat_test)
yhat.rf.bin <- if_else(yhat.rf.bin >= 0.5, 1, 0)
confusionMatrix(as.factor(yhat.rf.bin), as.factor(bin.test))
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	31	3
1	4	2

Accuracy : 0.825
95% CI : (0.6722, 0.9266)
No Information Rate : 0.875
P-Value [Acc > NIR] : 0.8809

Kappa : 0.2632

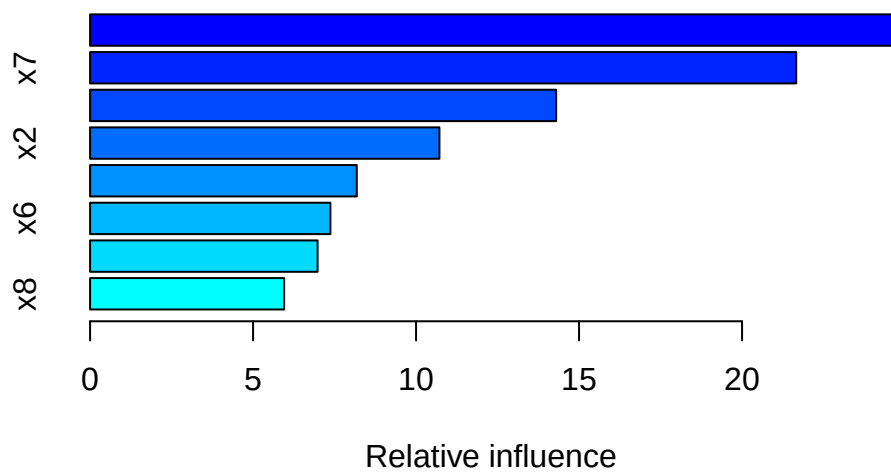
Mcnemar's Test P-Value : 1.0000

Sensitivity : 0.8857
Specificity : 0.4000
Pos Pred Value : 0.9118
Neg Pred Value : 0.3333
Prevalence : 0.8750
Detection Rate : 0.7750
Detection Prevalence : 0.8500
Balanced Accuracy : 0.6429

'Positive' Class : 0

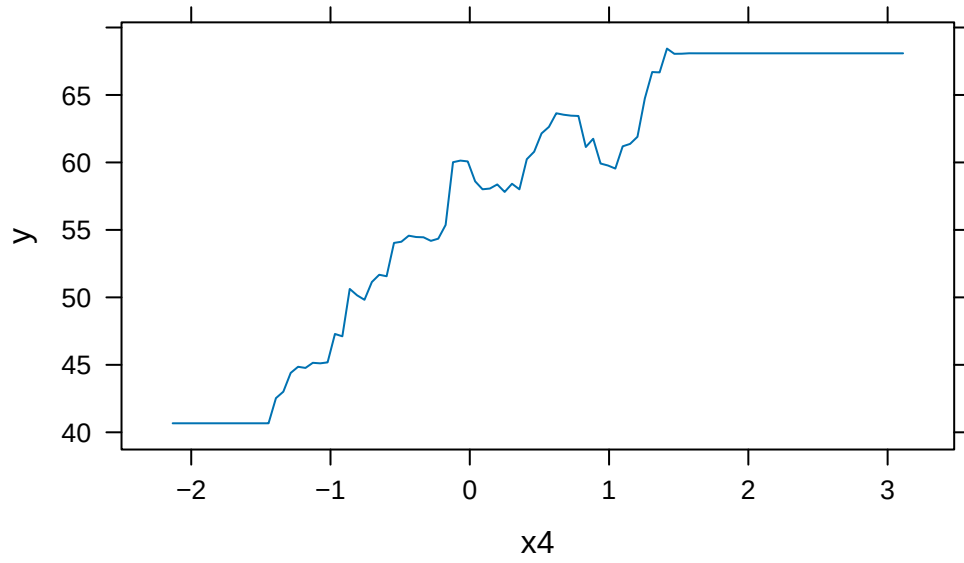
Boosting Numerical

```
set.seed(28)
boost.df <- gbm(y ~ ., data = train_df,
               distribution = "gaussian", n.trees = 5000,
               interaction.depth = 4)
summary(boost.df)
```

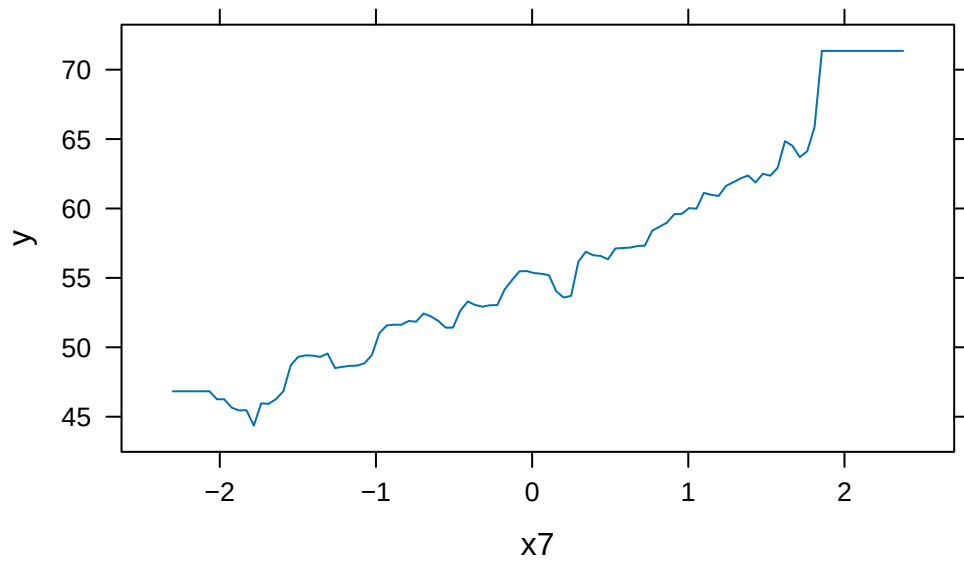


```
var    rel.inf
x4  x4 24.842100
x7  x7 21.660755
x5  x5 14.296910
x2  x2 10.716488
x3  x3  8.180380
x6  x6  7.371659
x1  x1  6.978931
x8  x8  5.952777
```

```
plot(boost.df, i = "x4")
```



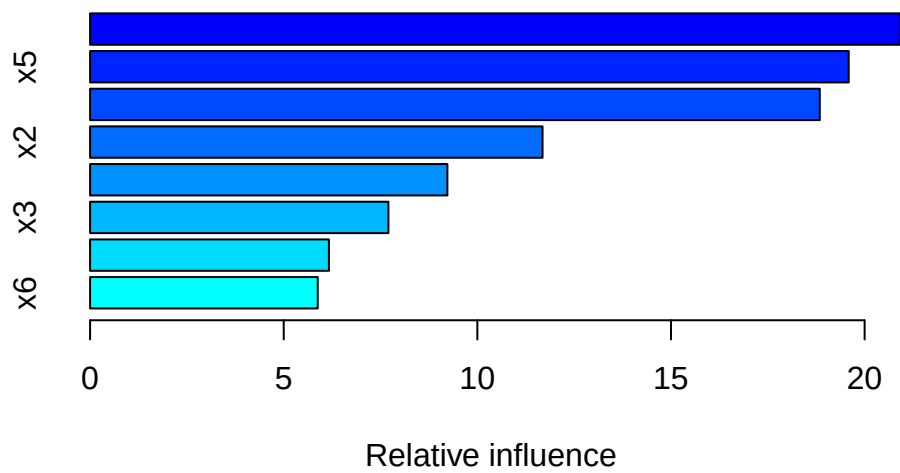
```
plot(boost.df, i = "x7")
```



```
yhat.boost <- predict(boost.df, newdata = test_df, n.trees = 5000)
mean((yhat.boost - df.test)^2)
```

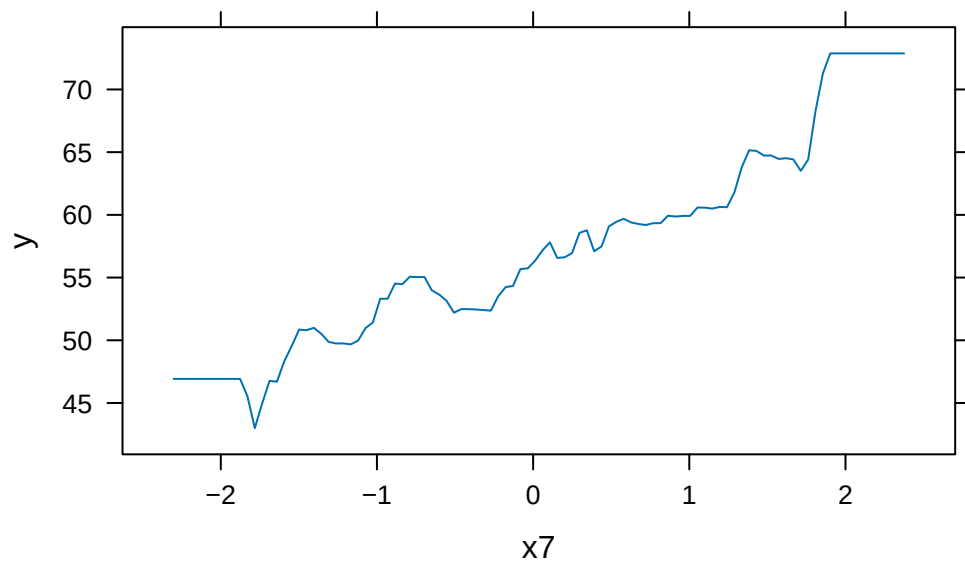
```
[1] 152.0158
```

```
boost.df.2 <- gbm(y ~ ., data = train_df,
  distribution = "gaussian", n.trees = 5000,
  interaction.depth = 4, shrinkage = 0.2, verbose = F)
summary(boost.df.2)
```

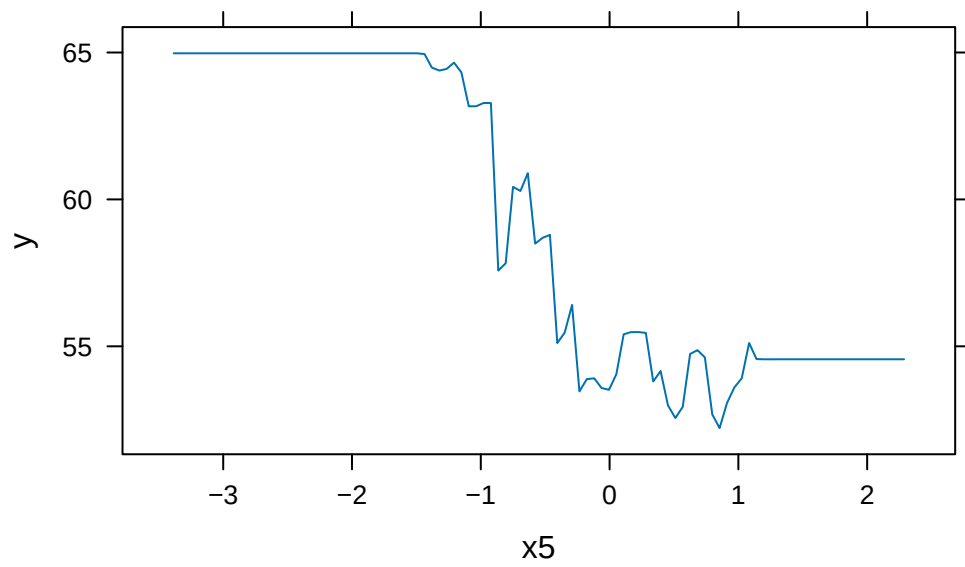


```
var    rel.inf
x7  x7 20.911134
x5  x5 19.589221
x4  x4 18.842276
x2  x2 11.683344
x1  x1  9.225559
x3  x3  7.703349
x8  x8  6.166627
x6  x6  5.878491
```

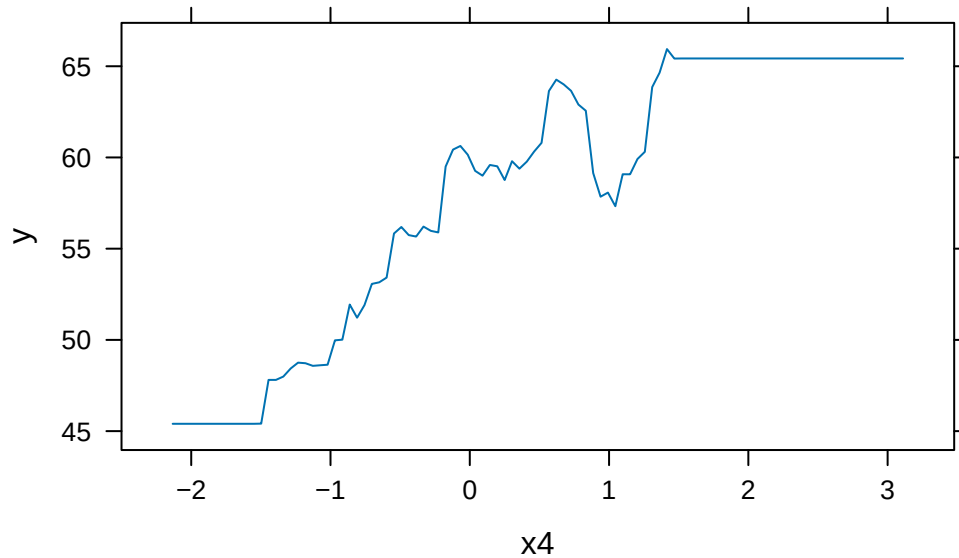
```
plot(boost.df.2, i = "x7")
```



```
plot(boost.df.2, i = "x5")
```



```
plot(boost.df.2, i = "x4")
```

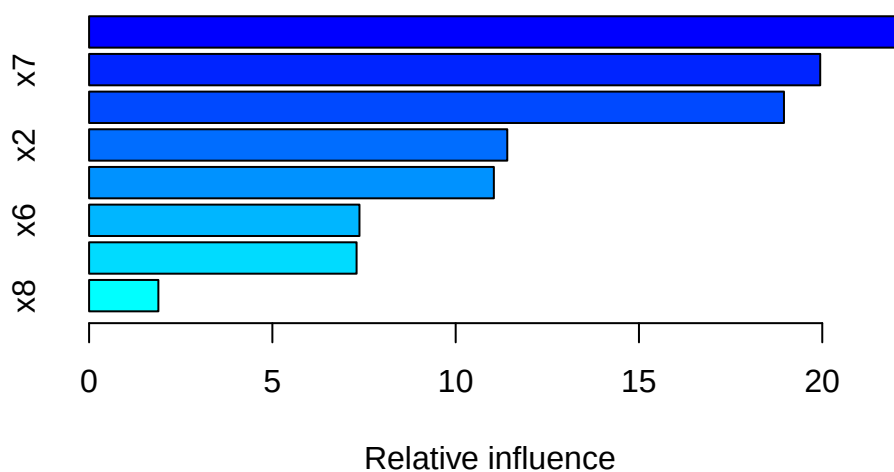


```
yhat.boost.2 <- predict(boost.df, newdata = test_df, n.trees = 5000)
mean((yhat.boost.2 - df.test)^2)
```

```
[1] 152.0158
```

Boosting Categorical

```
boost.df.bin <- gbm(ybin ~ ., data = df_cat_train,
  distribution = "bernoulli", n.trees = 5000,
  interaction.depth = 4, shrinkage = 0.2, verbose = F)
summary(boost.df.bin)
```

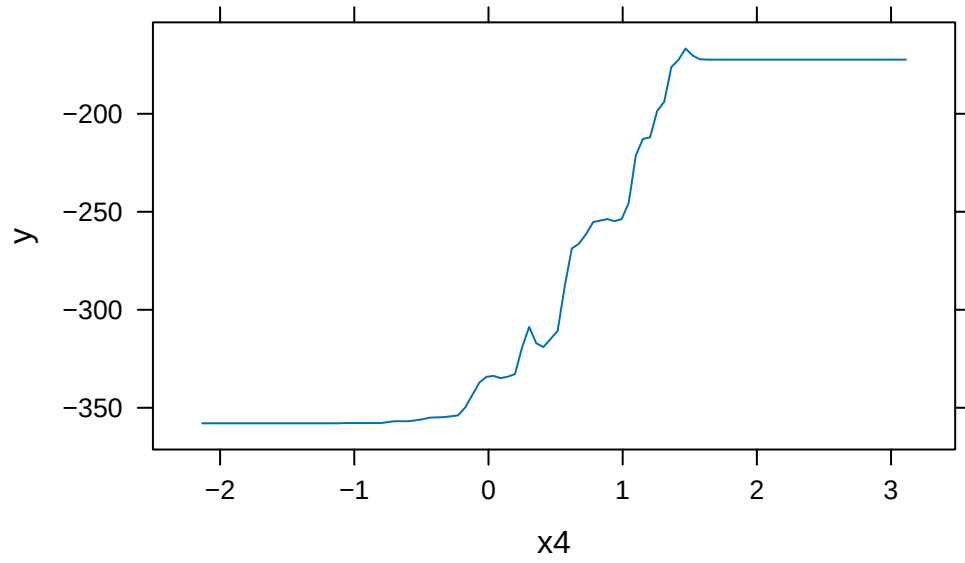


```

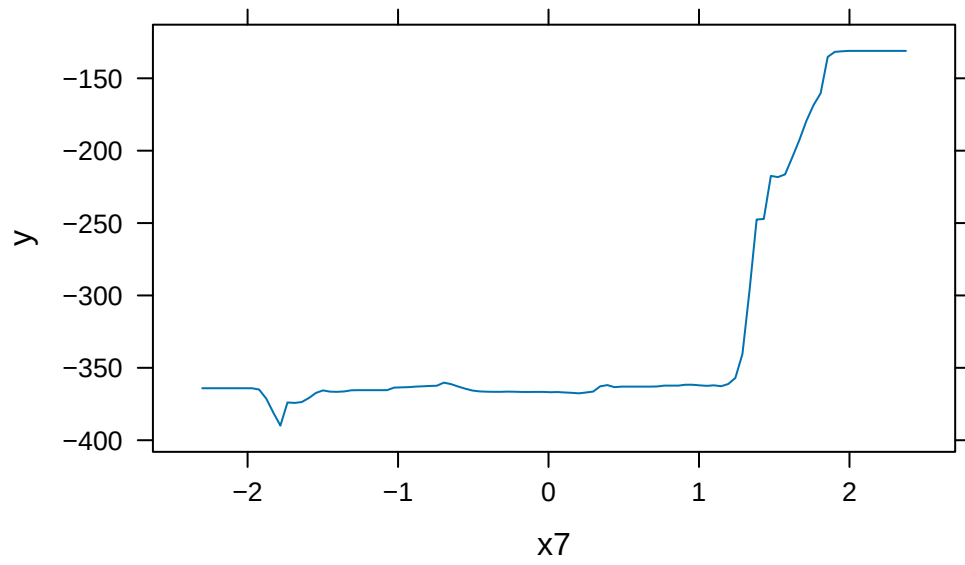
var    rel.inf
x4  x4 22.090620
x7  x7 19.943520
x5  x5 18.955164
x2  x2 11.407323
x3  x3 11.041424
x6  x6  7.375484
x1  x1  7.296916
x8  x8  1.889548

```

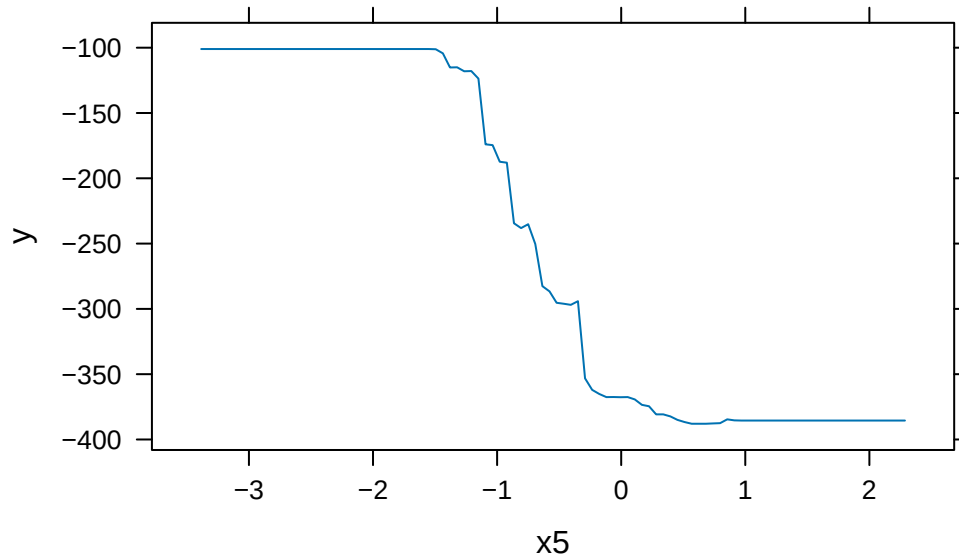
```
plot(boost.df.bin, i = "x4")
```



```
plot(boost.df.bin, i = "x7")
```



```
plot(boost.df.bin, i = "x5")
```



```
yhat.boost.bin <- predict(boost.df.bin, newdata = df_cat_test, n.trees = 5000)
yhat.boost.bin <- if_else(yhat.boost.bin >= .5, 1, 0)
confusionMatrix(as.factor(yhat.boost.bin), as.factor(bin.test))
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	32	3
1	3	2

Accuracy : 0.85
 95% CI : (0.7016, 0.9429)
 No Information Rate : 0.875
 P-Value [Acc > NIR] : 0.7725

Kappa : 0.3143

Mcnemar's Test P-Value : 1.0000

Sensitivity : 0.9143
Specificity : 0.4000
Pos Pred Value : 0.9143
Neg Pred Value : 0.4000
Prevalence : 0.8750
Detection Rate : 0.8000
Detection Prevalence : 0.8750
Balanced Accuracy : 0.6571

'Positive' Class : 0